

Solving multi-class problems by data-driven topology-preserving output codes



Junying Zhang^{a,*}, Xiaoxue Zhao^{a,b}, Lan Du^c

^a School of Computer Science and Technology, Xidian University, Xi'an 710071, P.R. China

^b Department of Computer Science, University College London, Gower Street, London WC1E6BT, UK

^c Electronics Engineering Institute, Xidian University, Xi'an 710071, P.R. China

ARTICLE INFO

Article history:

Received 2 June 2011

Received in revised form

15 May 2013

Accepted 20 May 2013

Communicated by G. Thimm

Available online 11 June 2013

Keywords:

Multi-class problem

Complexity

k-ary classifiers

Topology-order preservation

Error correcting output code

ABSTRACT

Aiming at decomposing a complex multi-class problem into fewer and simpler sub-problems to gain an overall classifier of low complexity, we propose a universal data-driven topology-preserving output code (TPOC) scheme, and a computationally efficient supervised circular learning algorithm (CLA) for the learning of the required TPOC map in the scheme. The scheme leads to a compact code and low complexity, and is an extension of binary, ternary and ECOC code. Experiments on Iris data, NCI data, octaphase-shift-keying data and handwritten digits reveal that the scheme substantially outperforms DECOC, one-against-all, natural coding and ECOC in using a less complex classifier with no loss or even enhanced generalization performance: the total number of support vectors is reduced greatly in SVM study and that of synaptic weights is greatly reduced (e.g., by 86% with training time reduced by 98% in MLP study in handwritten digit recognition problem); the total number of synaptic weights is further reduced by about one-fourth with less than one-hundredth loss of generalization performance when classifier complexities are assigned adaptive to the coding process. Finally, it is successfully applied to automatic target recognition based on a real measured radar data.

© 2013 Elsevier B.V. All rights reserved.

1. Introduction

One of the efficient ways for solving a complex multi-class classification problem is to code classes such that the problem can be decomposed to sub-problems each of which is simple to be solved. This is beneficial in many aspects. At first, each sub-problem can be solved by a simple classifier. Secondly, simple classifier can be with fewer parameters, making learning and test process efficient in computation and storage requirements and with less possibility of getting stuck into local minimum. Therefore, classification performance is easy to be guaranteed. Furthermore, boundaries obtained from simple classifiers will facilitate understanding and interpretation of decision rules. Other benefits include scalability, transparency [1,2], specific feature subset for different classifier and so forth.

In the perspective of complexity, the statement appeared that complexity is too difficult to deal with which greatly hinders the study of complexity in this approach. As a result, to our knowledge, no references focus on how to code classes

on complexity reduction. At least three reasons lead to the situation:

1. Complexity is data dependent. For different applications/classification problems, the scattering situation of data/each class of data in input space is greatly different. Some are scattered greatly, some are of great concentration, and some might be with local concentration but overall scattering, or vice versa. This leads to great differences in complexity of the required classifiers and/or decision boundaries.
2. Many factors on classifier affect complexity: classifier type (multilayer perceptron—MLP, support vector machine—SVM, etc.), classifier structural complexity (activation function of a neuron, number of hidden neurons in MLP, kernel type and kernel related parameter in SVM, etc.), learning algorithm and related parameter (e.g., learning rate), initialization, and so forth. This leads to great difficulty in studying complexity.
3. No strict complexity measure has been provided and can be applicable/adopted in this study. For example, the well known Kolmogorov complexity cannot be adopted due to its incomputability. VC dimension (for Vapnik–Chervonenkis dimension), as a measure of the capacity of a classification algorithm, is informally related to how complicated it can be. However, it is only meaningful in theoretical sense, while not applicable to this study. Akaike information criterion (AIC) is a

* Corresponding author. Tel.: +139 928 15979.

E-mail addresses: jy Zhang@mail.xidian.edu.cn, jy Zhang_xidian@qq.com (J. Zhang), x.zhao@cs.ucl.ac.uk (X. Zhao), du lan@mail.xidian.edu.cn (L. Du).

measure of the relative goodness of fit of a statistical model and its value provides a means for model selection. The closely related Bayesian information criterion (BIC) is also a criterion which can be used for model selection. However, they tell nothing about how well a model fits the data in an absolute sense. In practice one starts with a set of candidate models (or classifiers) and then finds their AIC/BIC values for getting the model that minimizes information loss. However, in this study, we do not have models, but aim at learning a code which is independent of models and leads to low complexity of models.

To our knowledge, no coding scheme is focusing on reducing complexity, but on problem decomposition without a concern to complexity: coding classes with error correction ability, and/or compact coding for reaching recognition performance. However, all these may not lead to low complexity. Two typical coding schemes are “one against all” (OAA) and “one against one” (OAO), which learn individual binary functions without any consideration of data topology; thus complexity is not in consideration. Error-correcting output code (ECOC) scheme, which has been extensively studied [3–12] and found in a diversity of applications in recent publications [13–21], is also data independent and hence complexity is not considered.

DECOC [7] and discriminant ECOC [5] are two recently developed compact-coding schemes which are data-driven and allow classes to organize in maximally discriminant sets, leading to compact code in some sense, which is the first promising step toward reducing complexity. However, compact code may not lead to low complexity, which will be seen from our experiment in Section 3.1. In addition, they may not be compact enough: DECOC requires (from the authors' experiments) the number of base learners only a little bit less than that of the OAO, and the discriminant ECOC, by embedding different tree structures to form a possible ECOC forest, has a restriction on generating a $K-1$ bit code for solving a K -class problem. In most previous work, e.g., OAA, OAO, ECOC, discriminant ECOC and ours [3,22], base learner is restricted to only binary one. All the above approaches can gain the code which can enhance the generalization performance successfully; however no concerns are paid to how complex classifier is used for such performance.

Using classifier of low complexity to reach high recognition performance is the goal. Complexity is not only significant in computation and storage requirement, but also affects the possibility of learning algorithm to get stuck to local minimum, especially for large scale complicated classification problem. How to classify classes really affects complexity greatly [3]. A lot of approaches indicate this, and some methods have been proposed, e.g., each sub-problem can focus on a different subset of input features [23], different parts of the input space [6], or different training samples [24,25]. However, these approaches are not in the framework of coding schemes.

Aiming at decomposing the problem into fewer and simpler sub-problems, or equivalently, learning a compact code which leads to low complexity, we propose a brand new data-driven topology-preserving output code (TPOC) scheme. The idea is to map the data into a latent small-sized discrete hypergrid, referred to as TPOC map, with two significant properties: *discriminative information preservation (DIP)* and *topology order preservation (TOP)*. The discrete hypergrid as the mapped space results in a code, the small size of the hypergrid leads the code to be compact, the DIP property makes the code be “meaningful” [4], and the TOP property results in the map of low non-linearity. All these together lead to low complexity of the overall classifier.

We propose a computationally efficient supervised circular learning algorithm (CLA) for learning the TPOC map. Self-organizing map (SOM) learning algorithm is incorporated due to

its topology-preserving property, which matches the required TOP property. Two user parameters are introduced for error-correcting ability and for using fewer classifiers. User can select one of them depending on his/her application requirement.

The obtained TPOC code is distinctive in the following characteristics: (1) it leads to low complexity; (2) it is data-driven, highly flexible, compact and classifier independent; (3) the code does not limit itself to any particular base learner: it can be either a k -ary one (where k is equal to or larger than 2) or a monolithic one. (4) the code, referred to as q-TPOC, can have error correcting ability.

The rest of the paper is organized as follows. Section 2 introduces the TPOC scheme starting from a simple example, then TPOC map definition and CLA algorithm, followed by an octaphase-shift-keying (eight-PSK) classification example. Section 3 demonstrates experimental results on the comparison of the TPOC with DECOC, OAA, random coding, natural coding, and ECOC, as well as comparison of TPOC and q-TPOC, on generalization performance and complexity, followed with an application of the TPOC to automatic target recognition for a real measured radar data. With the results showing great improvement by our scheme, Section 4 provides discussions on the advantages of the TPOC and relation to other related approaches. Finally, conclusions are given in Section 5.

2. Topology-preserving output code scheme

2.1. A first-place description

Firstly we describe the TPOC scheme with a simple example. Consider a 6-class problem in a two-dimensional feature space (see Fig. 1(a)), in which each class gathers around a grid-point of a 2×3 grid, $\{c_1, c_2, \dots, c_6\}$. We refer to the dataset as 2×3 -grid data for its special topological structure. The decision boundary from the OAA scheme is shown in Fig. 1(b), and the code matrix is listed as M_{OAA} in Table 1 (each column corresponds to a base learner). It is seen that the boundary is complex with many overlapping areas.

Yet we adopt a different coding strategy. For the grid-like topological property of the dataset, an apparently natural way to design classifiers is to directly assign them according to the grids. In detail, one binary classifier is designed to separate between $\{c_1, c_3, c_5\}$ and $\{c_2, c_4, c_6\}$, and another 3-ary classifier designed to separate $\{c_1, c_2\}$, $\{c_3, c_4\}$ and $\{c_5, c_6\}$. Or the 3-ary classifier can also be replaced by two binary classifiers separating between $\{c_1, c_2\}$ and the rest, and $\{c_5, c_6\}$ and the rest, respectively. The two corresponding code matrices are given in Table 1, labeled by M_{TPOC1} and M_{TPOC2} respectively, and in Fig. 1(d) and (e) respectively. It is clear that by harnessing the topological property of the grid-like data sets, these two strategies lead to fewer classifiers, and linear boundaries (see Fig. 1(c)), and thus result in fewer classifiers of lower complexities, i.e., an overall classifier of lower complexity, in solving the problem.

To see why such codes lead to lower complexity, we draw mapped space and decision boundary in mapped space in Fig. 1(d) and (e) for TPOC1 and TPOC2 respectively. Notice that code topology of TPOC1 and TPOC2 is same/similar to that of the dataset. This can be seen by comparing dataset given in Fig. 1(a) with the TPOC code given in Fig. 1(d) and (e) respectively. Whenever such code is obtained, decision boundary in mapped space can always be the least complex, e.g., solid lines in Fig. 1(d) and planes in Fig. 1(e). Hence finding a map whose code topology is same/similar to that of dataset is vital in leading to low complexity of the overall classifier.

In fact, in this example, only 3 binary classifiers in TPOC2/a binary and a ternary classifier in TPOC1 are used for this 6-class problem. Moreover, the classifiers in the schemes are all the simplest (linear). This means that even though some class vs the rest classes might be

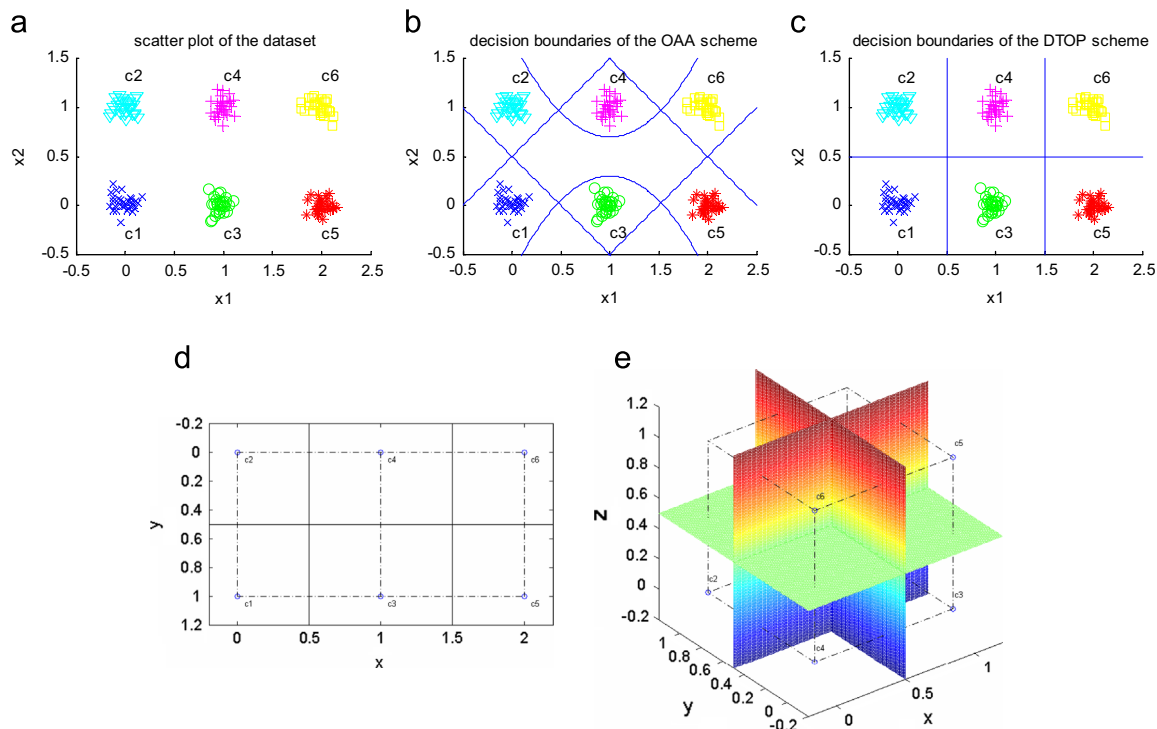


Fig. 1. Comparison between the conventional OAA scheme and the proposed TPOC scheme on complexity of decision boundaries. (a) A 2×3 -grid dataset; (b) decision boundary of OAA scheme; (c) optimal decision boundary by TPOC scheme; (d) code given by TPOC1 (circles on the grid points) and corresponding boundary (solid lines) in mapped space, and (e) code given by TPOC2 (circular grid points) and corresponding boundary (planes) in mapped space.

Table 1

Code matrices of OAA scheme and TPOC schemes for the 2×3 -grid data example.

	M_{OAA}						M_{TPOC1}		M_{TPOC2}		
	1-rest	2-rest	3-rest	4-rest	5-rest	6-rest	135-rest	12-34-56	135-rest	12-rest	56-rest
c_1	1	0	0	0	0	0	1	0	1	1	0
c_2	0	1	0	0	0	0	0	0	0	1	0
c_3	0	0	1	0	0	0	1	1	1	0	0
c_4	0	0	0	1	0	0	0	1	0	0	0
c_5	0	0	0	0	1	0	1	2	1	0	1
c_6	0	0	0	0	0	1	0	2	0	0	1

complicated in decision boundaries, some classes combined vs the rest can be very simple, the cause that even though less number of classifiers are used, decision boundary is kept the simplest: only when some classes are combined for separation from the rest, can the decision boundary be kept simple. Both are competitive to the DECOC which uses at least N binary classifiers ($N > K$), and to the discriminant ECOC which requires $K-1$ binary classifiers for a K -class problem, though both are also data-dependent: they are not compact enough in this example compared to the TPOC1 and TPOC2.

Note that two different clusters c_i and c_j can be of the same class. For example, with c_1 and c_2 datasets referring to a same class label, decision boundary could still be in linearity as depicted in Fig. 1(c), given that testing results of both c_1 and c_2 refer to the same class. Another example is the case of c_1 and c_4 datasets referring to the same class label. Though corresponding decision boundary of OAA for separating this class from the remaining classes will be much curved, it could still be the same as depicted in Fig. 1(c). In this sense, a class can be divided to several clusters each around a gridpoint. This indicates that clustering a class to clusters before discrimination of classes is generally helpful in using simple classifiers. For avoiding ambiguity, we always use clusters instead of classes in the following sections.

In the case of c_1 and c_2 datasets referring to the same class, the classifier corresponding to the first column bit (also called bit) in M_{TPOC1} and M_{TPOC2} can be learned with only samples labeled from c_3 through c_6 , by simply ignoring the ones labeled c_1 and c_2 . This indicates that the approach might generate a ternary code depicted in [11,21].

Enlightened by the distinct benefit with a *topology-based* coding strategy by this example, we aim to present a topology-preserving output code (TPOC) scheme by mapping cluster centers to a latent hypergrid, hoping to achieve low complexity of the overall classifier.

2.2. Definition

As a more accurate description, we define a TPOC map from cluster centers to a discrete latent hypergrid H of small size, which has the following properties:

1. *Discriminative information preservation (DIP)*: cluster centers of different classes in the input space are guaranteed to be mapped onto distinct gridpoints in the hypergrid.

2. *Topological order preservation (TOP)*: topological order is preserved as largely as possible, e.g., neighboring cluster centers in the input space are mapped to neighboring gridpoints in the mapped space.

The size of H refers to dimensionality of the mapped space and cardinality of each dimension for the H .

The joining of small hypergrid size, the DIP property and the TOP property results in low complexity: mapping to a small sized hypergrid leads to a compact code, the DIP property can guarantee that the code is “meaningful” [3], and the TOP property is vital in learning decision boundary which is simple. All these together lead to a “meaningful” compact code leading to simple decision boundaries. This is easily explained. Due to topological order preservation, a small sized TPOC map is the lowest complex map from the input space to the hypergrid where classes are kept discriminant and can be classified by simple (linear) decision boundaries in the mapped space. Intuitively, such decision boundaries, if they were available to be mapped back to input space, would approach the simplest decision boundary in input space, due to the lowest complexity of the map.

The TPOC map is computed from cluster centers and hence cluster analysis for each class is required. This is consistent with the statement in [26] that “sometimes cluster analysis may be used to separate the individual classes in the training set into subclasses”, and “consequently, simpler classifiers (e.g., linear) may be used and combined”.

2.3. TOP map learnt by self-organizing map (SOM)

Kohonen SOM (self organizing map) (Kohonen, 1982; 1995) is famous in learning a map with TOP property, one of the main reasons that it has been widely applied to many applications.

For learning the map with TOP property in this coding problem, we adopt the SOM with its output neurons set with the structure of a hypergrid. Standard SOM learning algorithm is employed to learn the map, which includes competitive process, cooperative process and adaptive process. It is these three joint processes that lead to the map with the TOP property. The competitive process is to determine the winning neuron $i(X)$ to be

$$i(X) = \arg \min_j \|X - W_j(n)\|, j = 1, 2, \dots, M \quad (1)$$

where $i(X)$ is the index of the winning neuron, X the newly input vector, and $W_j(n)$ the synaptic weight vector of neuron j at iteration n . The cooperative process is to define a topological neighborhood of the winning neuron as cooperative neurons. A typical choice for such a definition is the Gaussian function of

$$h_{j,i(X)}(n) = \exp\left(-\frac{d_{j,i}^2}{2\sigma^2(n)}\right) \quad (2)$$

where $d_{j,i(X)}$ is a measure of distance in mapped space between neuron j and neuron i and $\sigma(n)$ is the “effective width” of the topological neighborhood which is generally selected as an exponentially decreasing function with respect to iteration n . In the adaptive process, the winning neuron and its cooperative neurons are updated by the following equation when the newly input vector is X :

$$W_j(n+1) = W_j(n) + \eta(n)h_{j,i(X)}(n)(X - W_j(n)) \quad (3)$$

where the learning-rate parameter $\eta(n)$ takes values from $[0, 1]$ and decreases to zero as iteration n elapses.

2.4. Learning algorithm for a TPOC map

The task now is to learn a TPOC map. SOM can help learn a map with only the TOP property; however generally the DIP property cannot be guaranteed due to the unsupervised learning of the SOM. Here we introduce an SOM-based, heuristic supervised learning algorithm, referred to as circular learning algorithm (CLA, see the algorithm in the box) to learn a TPOC map.

Circular Learning Algorithm (CLA) for a TPOC map
Parameter: One of the following two parameters selected by a user:
 discriminant ratio threshold t ($0 < t < 1$) or
 the minimum Hamming distance threshold q (an integer not less than 3)
Input:
 An initial hypergrid H_0 as the mapped space; A set of cluster centers and their class labels.
Output:
 A TPOC map M_H on a hypergrid H of a suitable size
Learning process:
 Let $H = H_0$;
 Initialize the synaptic weight vectors of all the gridpoints at random;
 Set all the cluster centers to be mixed mapped ones;
Repeat
 Learn the map M_H with only mixed mapped cluster centers by conventional SOM learning algorithm;
 Calculate $\theta(M_H)$ and $\delta(M_H)$ from the obtained map M_H ;
 If $\theta(M_H) < t$ or $\delta(M_H) < q$
 Enlarge the hypergrid H ;
 Initialize the synaptic weight vectors of all the gridpoints on H at random; and
 Set all the cluster centers to be mixed mapped ones.
 Else
 Keep the mapped space H unchanged,
 Prohibit the update of synaptic weight vectors relating to discriminant gridpoints.
 End
Until $\theta(M_H) = 1$ or $\delta(M_H) \geq q$.

The novel idea of the CLA is below. One uses input data (cluster centers) to learn a hypergrid SOM of a small size. Due to the unsupervised learning of the SOM and possibly the too small size of the hypergrid, there might be gridpoints with different classes mapped onto, which disobeys the DIP property of the TPOC map. In this case, two choices can be made: learning the mixed gridpoints of the SOM with only the mixed cluster centers or enlarging the size of the hypergrid until some criterion is satisfied. The former enforces the map to be more discriminative for classes, and the latter allows for larger sized hypergrid which introduces more discrimination of classes or even error correction ability as ECOC functions, both under the constraint of topological order preservation (TOP). Such process proceeds until the DIP property is satisfied. The algorithm is described below.

Firstly, we obtain cluster centers of each class by using a clustering algorithm. Further steps are basically based on the obtained cluster centers. In this sense, the input of the algorithm is simply the cluster centers and the classes they belong to.

Secondly, we initialize a latent hypergrid $H = H_0$ as the mapped space. H_0 should be with small size in order to get comparatively few classifiers. The initialization can be experimental or subjected to some prior knowledge. For example, in cases without prior knowledge, the hypergrid can be simply initialized to be a $2 \times 2 \times \dots \times 2$ -hypercube, where the dimension of the hypercube is $\lceil \log_2 K \rceil$ where K is the number of classes in the problem, and in case with some prior knowledge, a more specific hypergrid can be initiated according to the knowledge. Then, we apply SOM learning algorithm to obtain the map M_H from cluster centers to gridpoints of H which possesses the TOP property. However, a simple application of SOM for this mapping may result in mixed mapping, i.e., cluster centers of different classes are mapped onto a same gridpoint, which disobeys the DIP property of the TPOC map.

Table 2

Comparison of the TPOC scheme and the DECOC scheme using binary linear SVMs for Iris data and NCI data.

	DECOC				TPOC			
	Training time (s)	# of SVMs	Pc (%)	SVs	Training time (s)	# of SVMs	Pc (%)	SVs
Iris data (3-class problem)	0.141	6(3 1ovs+3 2ovs)	90.67	253.6	0.078	2	96.00	39.1
NCI data (4-class problem)	0.220	6(4 1ovs+2 2ovs)	95.31	93.4	0.125	3	96.88	50.0

To avoid such mapping, normally a second SOM learning should be tried, or the hypergrid should be enlarged or reshaped. Typically, enlarging the hypergrid to a large size will avoid mixed mapping as described above. However, it also demands more classifiers, which makes the problem more complex. The opposite is that with a too small sized hypergrid, even the DIP property cannot be guaranteed, which makes the coding not “meaningful”. Therefore, we should stretch a balance between the number of the classifiers (i.e., the dimensionality of the hypergrid), and the complexity of each classifier.

To this end, we introduce a user parameter as the tradeoff between the number and the complexity of classifiers. t ($0 < t \leq 1$) as the threshold of ratio $\theta(M_H)$ (the ratio of the number of discriminatively mapped classes over the total number of classes) can be one of the choices of the balance. With the user-determined t , the program decides whether to retry on the same hypergrid, or enlarge the hypergrid. For example, when $\theta(M_H) < t$ which means too many mixed mapping cases, the size of hypergrid is enlarged, and the SOM learning algorithm is applied to the updated hypergrid. Else, the hypergrid is kept the same but SOM is retried on this hypergrid. For preserving the topology as largely as possible (the TOP property), we keep the learnt synaptic weight vectors relating to discriminative gridpoints unchanged, with others to update as usual, until no mixed mapping occurs, i.e., synaptic weight vector of a gridpoint is updated simply by

$$W_j(n+1) = \begin{cases} W_j(n) + \eta(n)h_{j,i(X)}(n)(X - W_j(n)), & \text{if } j \text{ is a mixed gridpoint;} \\ W_j(n), & \text{otherwise.} \end{cases} \quad (4)$$

where X is only the indiscriminant input vectors with the winning neuron $i(X)$ and the neighborhood function $h_{j,i(X)}(n)$ still given in Eqs. (1) and (2) respectively. The program loops until a reasonable mapping is obtained.

Yet another choice is also available as a user parameter. q (an integer) can be predetermined as the threshold of minimum Hamming distance $\delta(M_H)$ between pairs of codewords. The hypergrid is enlarged until $\delta(M_H)$ is equal to or larger than q . Hence, q represents the user defined smallest Hamming distance among codewords. With this parameter introduced, error-correcting ability is predetermined by the user.

The condition for leaving the loop is $\theta(M_H) = 1$ when t is the setting, and it is $\delta(M_H) \geq q$ when q is the setting. The former indicates that the learnt map (code) is discriminative to all classes, and the latter adds an additional user predetermined error-correcting ability.

For clear presentation, in the following parts, we denote the second type TPOC with q as user parameter to be q -TPOC. The two choices of user parameters are actually not conflicted. They both aim to provide a payoff for the number and complexity of classifiers, but from different perspectives. User can provide t or q according to his/her application requirement, which demands fewer classifiers or for error correction, both using simple classifiers.

When the TPOC map is obtained, from the map, similar to M_{TPOC1} and M_{TPOC2} in Table 2 for the 2×3 -grid data in Section 2.1, two available intuitive code solutions for coding classes can be

derived, using monolithic k -ary classifiers (k might be different for different classifiers) and using only binary classifiers, which are given in Appendix A1.

2.4.1. An example

As an example, we demonstrate TPOC scheme in solving a classification problem on an octaphase-shift-keying (eight-PSK) pattern space. We compare our result with the results by OAA, natural coding, and Gray-coding schemes presented by Sebal and Bucklew [3].

The scatter plot of the 2-dimensional samples in the eight-PSK dataset and the decision boundaries obtained by binary SVMs with polynomial kernel $K(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1 \cdot \mathbf{x}_2 + 1)^d$ of order d are drawn in Fig. 3 for the respective coding schemes.

By initializing the hypergrid to be $2 \times 2 \times 2$, the three TPOC schemes, i.e., $2 \times 2 \times 2$ -TPOC, $2 \times 2 \times 2 \times 2$ -TPOC and the 3×3 -TPOC, were obtained by the CLA, with the setting of $t=80\%$, the setting of $t=1$ with enlarging of the hypergrid in dimensionality, and the setting of $t=1$ with enlarging of the hypergrid in cardinality of each dimension respectively. The corresponding coding process of the CLA is shown in Fig. 2(a), (b) and (c).

For the $2 \times 2 \times 2$ -hypercube setting, since the SOM learning algorithm (i.e., the first run on the “repeat” circle in the CLA) converges to the coding shown in Fig. 2(a1) in which mixing codes exist, i.e., the code of 100 and the code of 001, both not discriminative to C1 and C2, and to C6 and C7 respectively, the CLA learns the SOM further (i.e., the second run on the “repeat” circle) by prohibiting the update of synaptic weight vectors relating to discriminant nodes (codes) of 000, 011, 110, and 111. The converged coding is shown in Fig. 2(a2). Since the DIP property is satisfied, the CLA terminates with the final coding result shown in Fig. 2(a2). For the $2 \times 2 \times 2 \times 2$ -hypercube and the 3×3 -grid settings, the SOM learning algorithm (the first run on the “repeat” circle in the CLA) converges directly to the coding shown in Fig. 2(b) and (c) respectively, where the DIP property is already satisfied.

Consider complexity of the decision boundaries. For an SVM with polynomial kernels, its complexity can be represented by polynomial order d and the number of support vectors. For solving a problem with multiple SVMs with the same order polynomial kernels, the total number of support vectors required, referred to as SVs,¹ can be used as the measure of the complexity of the overall classifier. From Fig. 3, only two schemes, $2 \times 2 \times 2 \times 2$ -TPOC and the 3×3 -TPOC shown in Fig. 3(b) and (c) respectively, achieve both linear decision boundary ($d=1$) and fewer SVs than others. The $2 \times 2 \times 2$ -TPOC shown in Fig. 3(a) also leads to comparatively low complexity. The TPOC schemes are superior to the other schemes on complexity of the overall classifier, and it is not sensitive to hypergrid configuration or parameter setting as long as t is set large enough as presented in Fig. 3(b) and (c).

¹ In SVs, support vectors appearing in separate SVMs are counted according to the times of their appearance. It was stated by Burges [36] that a great reduction of SVs indicates a great reduction of the computation and storage requirements.

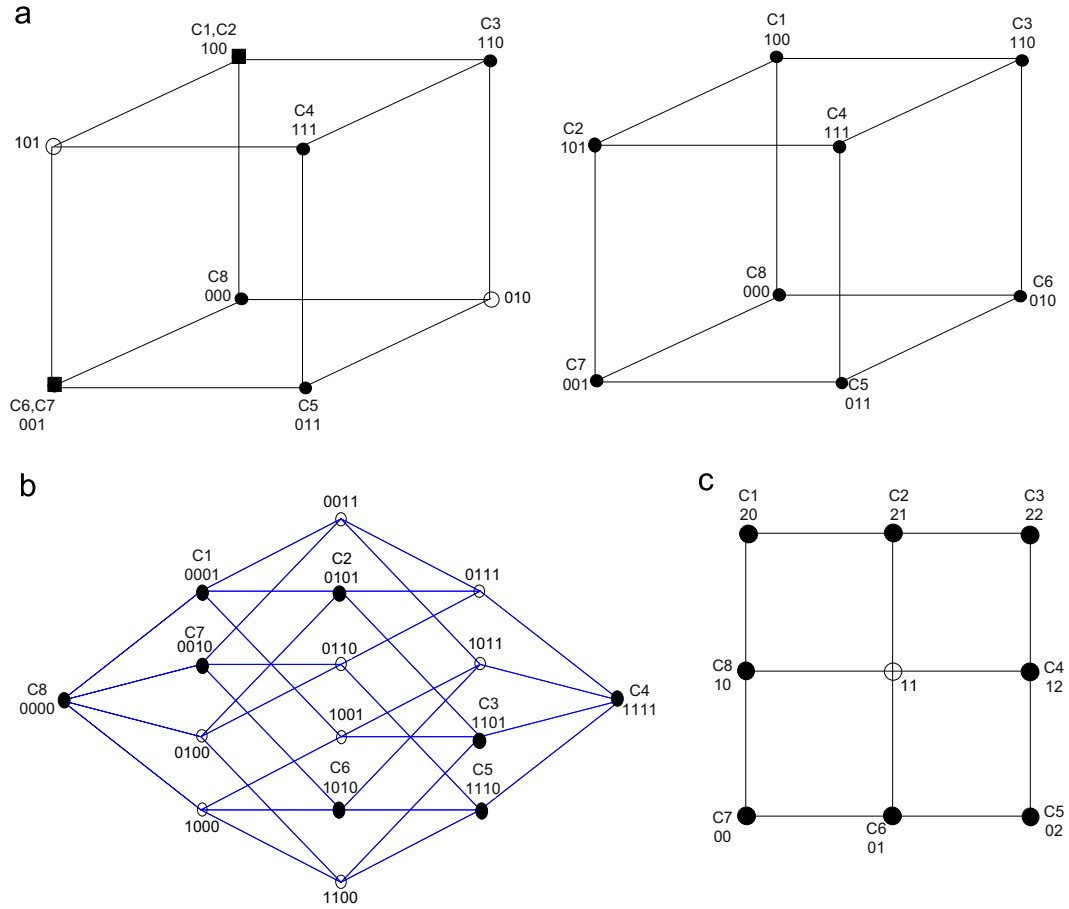


Fig. 2. The coding process of the CLA with (a) $2 \times 2 \times 2$ -, (b) $2 \times 2 \times 2$ - and (c) 3×3 -hypergrid settings for the 8-PSK dataset, where the node with vacuum circle, solid circle and solid square indicates the code not assigned to any class, assigned to some single class and assigned to multiple classes. (a1) and (a2) show the converged SOM after the first and the second run of SOM in CLA; (b) and (c) show the one after the first run of the SOM in CLA.

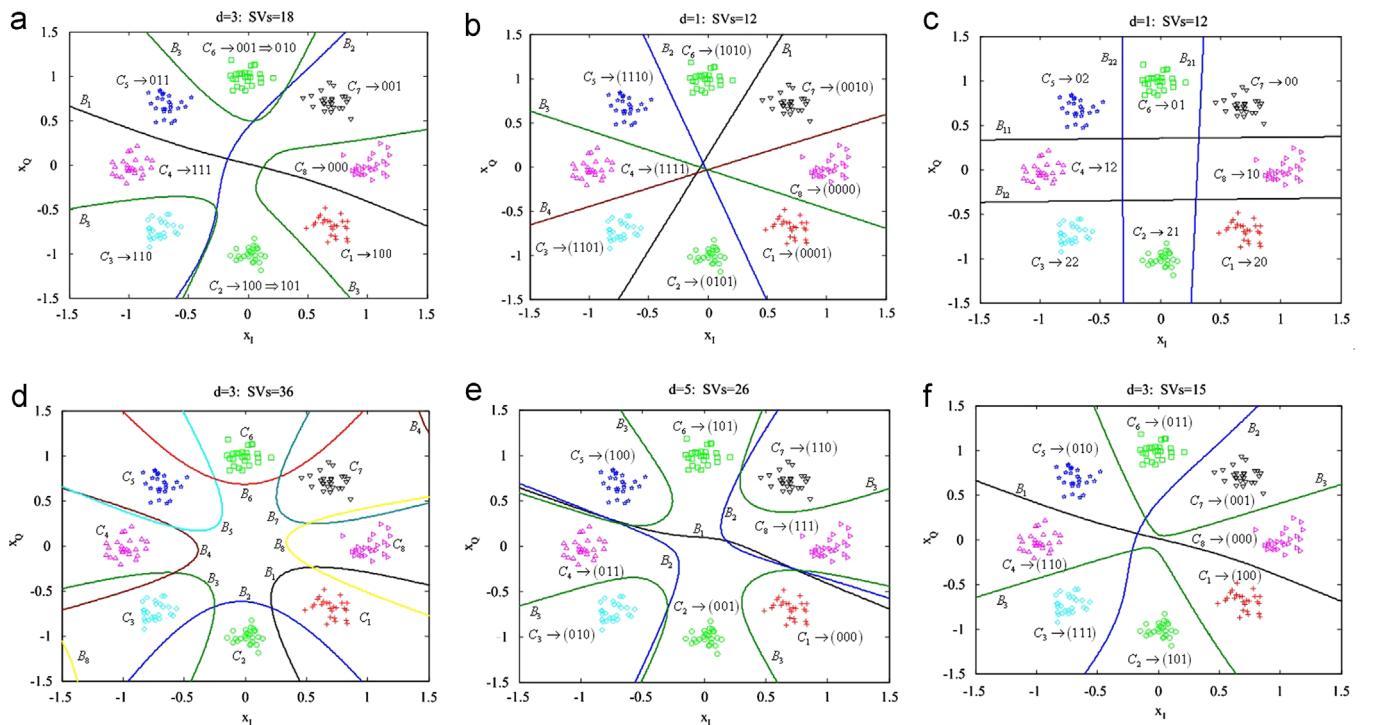


Fig. 3. Decision boundaries of binary SVMs based on different code representation schemes for the eight-PSK dataset: (a) $2 \times 2 \times 2$ -TPOC scheme, (b) $2 \times 2 \times 2$ -TPOC scheme, (c) 3×3 -TPOC scheme, (d) OAA scheme, (e) natural coding scheme and (f) gray coding scheme.

Now we discuss code adjustment in CLA learning process. In Fig. 3(b) and (c), there exists no code adjustment; correspondingly, the decision boundaries obtained are all simple (linear). However, there are code adjustments $001 \rightarrow 010$ and $100 \rightarrow 101$ shown in Fig. 3(a). No bit adjustment in the first bit ($0 \rightarrow 0$, $1 \rightarrow 1$) leads to simple decision boundary B_1 (almost linear, $d = 1$); the second bit adjustment ($0 \rightarrow 1$, $0 \rightarrow 0$) in fact compensates the non-linearity of the TPOC map, and leads to simple decision boundary B_2 being almost linear ($d = 1$); the third bit adjustment ($1 \rightarrow 0$, $0 \rightarrow 1$) enforces non-linearity in the TPOC map, and therefore leads to decision boundary B_3 being more complicated ($d = 3$). Hence, it seems available to have an adaptive choice on classifier parameters relating to complexity: the complexity of the non-adjusted-bit classifier can be assigned lower than that of adjusted-bit classifier. Validation experiments on this point will be given in Section 3.5.

3. Experiments and results

The aim of the experiments is to root out the potentials of the TPOC scheme on complexity compared with other typical coding schemes; hence a thorough comparison of the TPOC with other typical coding schemes on vast benchmark problems and real application problems is out of the extent of this paper, and kept open to researchers.

To this end, we took limited datasets as examples while studying diverse potentials of the TPOC scheme over some typical coding schemes: classifier complexity, error correcting ability, and possibility of adaptive assignment of classifier complexity. Though a lot of papers used the same datasets for experiments and some gained higher recognition performance, we did not cite them here, for they generally did not mention the complexity of the overall classifier used.

Experiment for comparison of TPOC and DECOC is given on Iris data and NCI data (Subsection 3.1). Then experiments on handwritten digits recognition are given for comparison of the proposed TPOC scheme with OAA (Section 3.2), with random coding and natural coding (Section 3.3), with q-TPOC and ECOC (Section 3.4), and comparison of the TPOC with and without adaptive assignment of classifier complexities (Section 3.5). At last, the proposed TPOC scheme was applied for automatic target recognition based on a real measured radar data (Section 3.6).

Since obtained code is classifier independent for all these schemes, we arbitrarily used a monolithic MLP, or base learners (k -ary MLPs or binary SVMs) as an overall classifier. Each bit of the code is used to label the output of a base learner or a codeword is used for labeling the output of a monolithic classifier, with details discussed in related context. In using base learners case, nearest neighbor decoding is selected in all the schemes for no rejection region [11,28].

Comparison is on generalization performance and complexity of the obtained overall classifier to see if the TPOC scheme can realize low complexity with no reduction on generalization performance. Complexity is measured by SVs when binary SVMs are used or the total number of parameters (synaptic weights) when MLPs or a monolithic MLP is used. Generalization performance is measured by recognition rate $P_c(\%)$ on test data. For avoiding local minimum of the back-propagation learning algorithm for an MLP, 5 runs were performed and the best one was selected.

3.1. Comparison of the TPOC with DECOC

Potential of the TPOC scheme over some up-to-date coding scheme, e.g., DECOC, can be seen from our experiment on Iris and NCI data.

DECOC scheme designs an effective and compact code. It uses 1-versus-others binary classifiers, 2-versus-others ones and 3-versus-others ones, simply denoted by 1vo, 2vo and 3vo, for a K -class problem. The number of 1vo learners is K , and all 2vo and 3vo base learners were learnt, and those with the highest confidence scores are selected. The number of 2vo and 3vo base learners are determined by iterative search and cross validation. For the search to start, the proportion p_{3vo} of the 3vo learners in all base learners is prespecified, which is 0 for $K < 5$ and $p_{3vo} \in [0, 1 - K/N]$, where N is the length of the code.

We simply used Iris data and NCI data as examples for comparison. Iris data has 150 samples in 4-dim space belonging to 3 classes, and NCI data [29] includes 64 samples within 4 classes each with expression levels of 2308 genes. Due to the very high dimensionality of the space for the NCI data (i.e., 2308), we selected 6 genes by using the mRMR feature selection method [30]. Linear binary SVM was used as a base learner and 10-fold cross validation was used for getting P_c for both datasets. In TPOC scheme in this experiment, each class was set to be a cluster and 2×2 -TPOC for Iris and $2 \times 2 \times 2$ -TPOC for NCI data were experimented respectively, reaching a 2-bit code and a 3-bit code. In DECOC, we consider N SVMs with N set being 6 for both datasets and hence 6-bit DECOC code is obtained. The number of classes in the two problems are all less than 5; hence according to DECOC, we set parameter $p_{3ov} = 0$ for both the datasets.

As declared, DECOC is a compact code. However, seen from the result given in Table 2, TPOC is more compact: TPOC is only 2-bit/3-bit code for the Iris/NCI data; however DECOC is a 6-bit code for both the datasets. In addition, the TPOC leads to low complexity of the overall classifier (less SVs) than the DECOC: the SVs reduced by about one sixth/a half for the Iris/NCI data. Though with fewer and lower complex classifiers, the recognition performance is not decreased but even enhanced, the power of the TPOC scheme for solving a problem with fewer classifiers of low complexity.

This experimental result indicates that focusing on compact code (like DECOC) may not lead to low complexity, while focusing on complexity in some sense will formulate a compact code.

3.2. Comparison between TPOC and OAA

In this section we compare TPOC and OAA on the digit dataset.

The digit dataset consisting of handwritten numerals ("0"–"9") is extracted from a collection of Dutch utility maps.² 64 Karhunen–Loève features were selected for recognition. In experiments, we always used the same subset of 1000 samples for testing and the subset of the remaining 1000 patterns for training.

As the pre-procedure of the TPOC scheme, any clustering algorithm can be used for clustering data into clusters. Here we adopt rival penalized competitive learning (RPCL) algorithm for it does not need prior knowledge on the number of clusters [39]. For handwritten digits, we initialized RPCL with $J=4$ (predetermined number of clusters in RPCL), and detected the numbers of clusters for each handwritten digit being 2, 2, 3, 2, 3, 3, 3, 3, 2 and 2 for class 0–9 correspondingly.

Since 64 Karhunen–Loève features were selected and 10 handwritten numerals ("0"–"9") were to be recognized, the input space is 64 dimensions and the data in the space falls into 10 classes. Two simple base MLPs based on the TPOC scheme and a monolithic complex MLP based on the OAA scheme were employed for comparison.

The OAA scheme leads to 10 binary classifiers each of which separates a digit from the remaining ones. Using a monolithic MLP to

² The dataset is available through Irvine Machine Learning Repository (www.ics.uci.edu/~mlearn/MLRepository.html).

implement it, the MLP is with 64 inputs and 10 outputs, where each output neuron corresponds to a binary classifier separating the digit from the remaining ones, a good match to the OAA scheme. Separating the 10 handwritten digits with a *monolithic* MLP must be nontrivial. We set the number of hidden neurons to be somewhat large, say 20 in this experiment. Hence the monolithic classifier is 64-20-10 MLP.

For using the TPOC scheme for the task, we set the TPOC map to be a 4×4 grid and learnt it by the proposed CLA with $t=80\%$. After the learning, 2-bit 4-ary code was obtained; hence two 4-ary classifiers were required. By the knowledge that the TPOC scheme can produce code which leads to low complex classifier, a simple MLP, i.e., the MLP with only 3 hidden neurons, was used as each classifier. Hence we used two 64-3-4 MLPs for the recognition problem, with each MLP output labeled with corresponding bit of the code.

The results of both the OAA and the TPOC are shown in Table 3. It shows that the storage and computation requirement of the TPOC scheme (211 parameters and 211 multiplications for each 64-3-4 MLP) is much less than those of the 64-20-10 MLP (1510 parameters and 1510 multiplications). The total number of synaptic weights of the TPOC is reduced by 86% compared with that of the monolithic MLP and the time for training the two 64-3-4 MLPs consecutively is 8.57 mins, only 2% of that for training the 64-20-10 MLP, 7.16 h. With such small storage and training time, the P_c is not decreased but even increased compared with that of the monolithic MLP. This experiment demonstrates that the computation complexity and storage requirement can be drastically

reduced without any loss of recognition performance by using the TPOC scheme.

Fig. 4(a) and (b) shows the scatter plots of the hidden layer outputs (3 dimensions) of the two 64-3-4 MLPs respectively, and Fig. 4(c) the scatter plot of the hidden layer outputs (20 dimensions) projected onto the first three weighted pairwise Fisher principal directions [31]. It can be seen that each of the 4-class recognition sub-problems shown in Fig. 4(a) and (b) is much easier to be solved than the 10-class problem shown in Fig. 4(c). This experiment indicates that a suitable coding of classes can really reduce a complex classification problem to drastically simple classification sub-problems.

3.3. Comparison between TPOC, random code and natural code

For the 10-class digit recognition problem, only 4 output neurons of a monolithic MLP are enough for coding the 10 classes. Hence we consider only 4-bit code for labeling the output of the MLP. The coding schemes are $2 \times 2 \times 2 \times 2$ -TPOC scheme, $2 \times 2 \times 3$ -TPOC, 4-bit natural code and 4-bit binary random code. The number of hidden neurons h was set to be the same for each scheme (hence the MLPs are with a same complexity), being 4, 8, 12, ..., 80 respectively. In random coding scheme, we generated “meaningful” code randomly and arbitrarily assigned to classes 20 times to get error-bar plot of the result. The obtained P_c with respect to h is shown in Fig. 5.

From Fig. 5, we have the following observations: at first, coding scheme really affects recognition performance; secondly, recognition performance of the TPOC is *not seriously sensitive* to the hypergrid structure ($2 \times 2 \times 2 \times 2$ and $2 \times 2 \times 3$); furthermore,

Table 3
The recognition result of a monolithic 10-ary MLP scheme and a TPOC scheme using less-sized MLPs for the set of handwritten digits.

OAA scheme, one 10-ary 64-20-10 MLP		
Storage requirement	Hidden layer	Weight vector: 64×20 , Bias vector: 1×20
	Output layer	Weight vector: 20×10 , Bias vector: 1×10
Training time P_c (%)		
4 × 4-TPOC scheme, two 64-3-4 MLPs		
The 4 × 4-TPOC code		
“0” → 03; “1” → 00; “2” → 30; “3” → 21; “4” → 23; “5” → 11; “6” → 13; “7” → 20; “8” → 02; “9” → 01.		
Storage requirement	Hidden layer	Weight vector: 64×3 , Bias vector: 1×3
	Output layer	Weight vector: 3×4 , Bias vector: 1×4
Training time P_c (%)		
514.6090 s = 8.58 mins = 0.14 h 86.0		

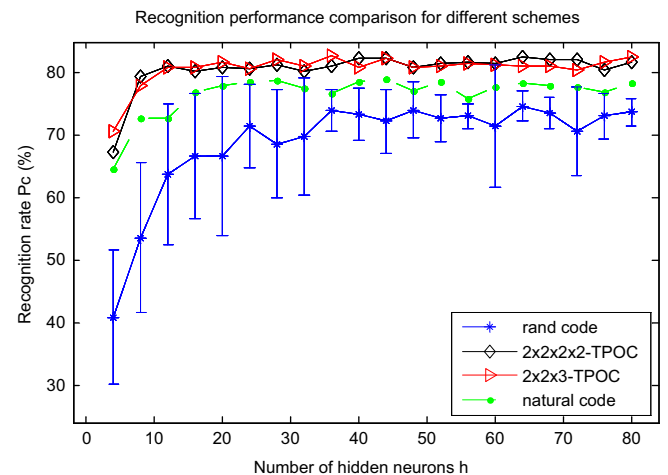


Fig. 5. Recognition performance of the TPOC scheme and comparison with natural coding scheme and random coding scheme for the set of handwritten digits.

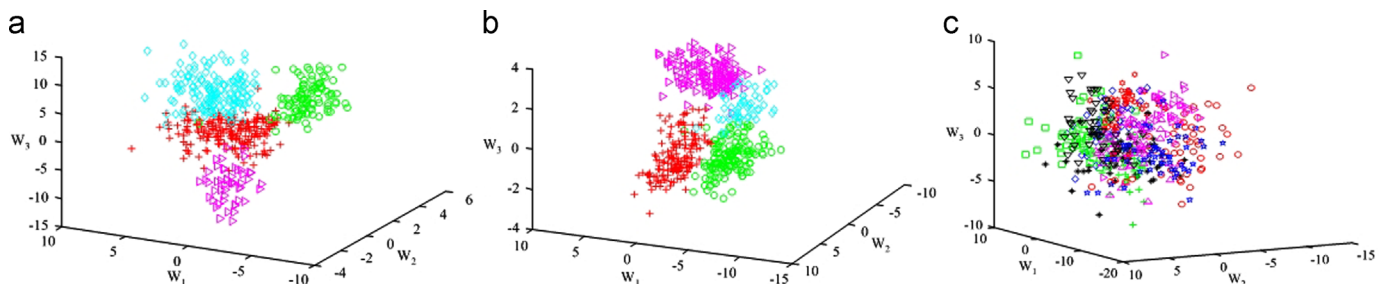


Fig. 4. Scatter plots of the training samples in the hidden layer's outputs for the set of handwritten digits.

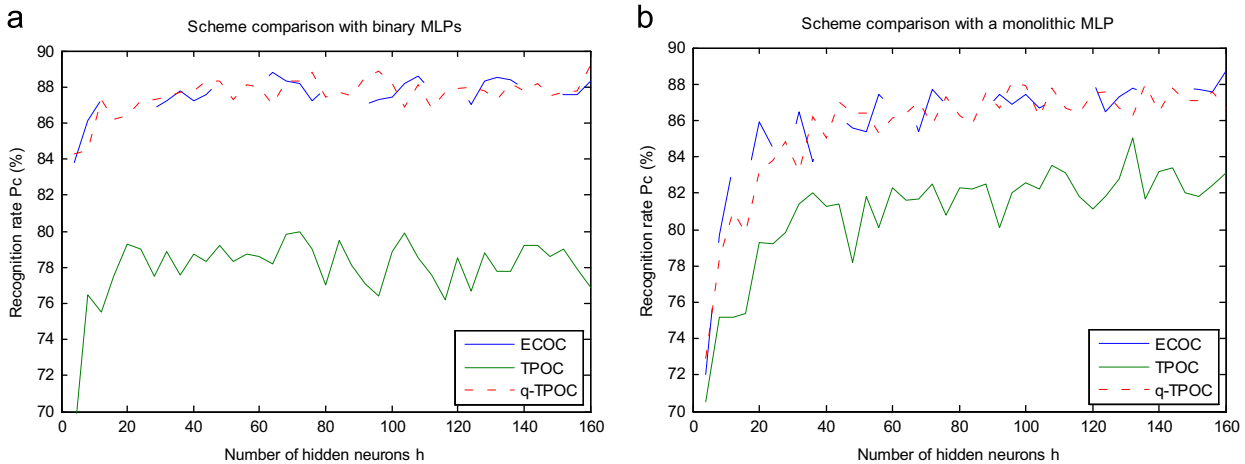


Fig. 6. Comparison between 4-bit TPOC, 10-bit q -TPOC and 15-bit ECOC schemes on recognition performance, when the schemes were implemented with (a) binary MLPs and (b) a monolithic MLP.

both the $2 \times 2 \times 2 \times 2$ - and $2 \times 2 \times 3$ -TPOC outperform natural code and random code in recognition performance for same complexity of the classifier: P_c of the TPOC increases about 8% compared with the average one of random codes and it increases about 3% compared with natural code.

3.4. Comparison among TPOC, q -TPOC and ECOC schemes

Binary coding schemes were used for comparison. The schemes are 4-bit TPOC, 10-bit q -TPOC and 15-bit ECOC. The 4-bit TPOC comes from an initial $2 \times 2 \times 2 \times 2$ -hypercube learnt by CLA with $t=80\%$, the adopted 15-bit ECOC comes from the Table 3 of Ref. [3], and the 10-bit q -TPOC comes from an initial $2 \times 2 \times 2 \times 2$ -hypercube learnt by CLA with $q=4$ and enlarging always in dimension of the hypercube. The 4-bit TPOC, 10-bit TPOC and the 15-bit ECOC has no, one and up to 3 errors correction ability, and are denoted simply as TPOC, q -TPOC and ECOC respectively in this subsection.

The schemes were applied to a monolithic MLP and binary base MLPs respectively to see consistency of the result on different classifiers. If the result is consistent, it indicates that the result is independent of classifier setting. Comparison is among TPOC, q -TPOC and ECOC.

Suppose the code is a k -bit code. When a monolithic MLP is used, a 64 - h - k MLP was employed, with output labeled by the codeword of a class. When binary base MLPs are utilized, k binary MLPs, i.e., 64 - h -1 MLPs, were employed. The number of hidden neurons h was selected uniformly from 4 to 160 with space of 4. The obtained P_c is shown in Fig. 6.

At first, results from binary MLPs and from a monolithic MLP are greatly consistent, seen from a comparison of curves in Fig. 6(a) and (b). This indicates that the result is independent of classifier setting.

Secondly, as seen from Fig. 6, the q -TPOC improves P_c greatly (more than 5%) compared with the TPOC, attributing to its error-correcting ability.

Moreover, comparing the q -TPOC and ECOC, the former is with 1 error correction ability, while the latter is with up to 3 errors correction ability; the former was implemented by an MLP with 10 outputs/by 10 binary MLPs, while the latter was implemented with an MLP with 15 outputs/by 15 binary MLPs. This indicates that both the complexity and the error correction ability of the overall classifier obtained from the q -TPOC are much less than those of the ECOC. However, as seen from Fig. 6, the P_c obtained by the q -TPOC reaches approximately the same as that by the ECOC. This reinforces the great potential of q -TPOC for getting high recognition performance with a low complex overall classifier.

3.5. Comparison of TPOC schemes with and without adaptive assignment of classifier complexities

What was observed from our experiments is that the P_c 's of non-adjusted-bit classifiers are generally higher than those of the adjusted-bit ones. This suggests possibility of using adaptive complexity for different classifier in the proposed TPOC scheme. In this experiment, we compare recognition performance between same complex MLPs and adaptively complex MLPs. The $2 \times 2 \times 2 \times 2$ -TPOC scheme was employed, and hence 4 binary MLPs were used for the problem. We used 4 binary MLPs in two manners: the number of hidden neurons h is fixed and it is adaptively assigned. In the first situation, h was set fixed for all the four MLPs, being from 2 to 20; in the second situation, h was set fixed from 2 to 20 for each adjusted-bit classifier, and was set to be a half of it, i.e., from 1 to 10 for each non-adjusted-bit classifier (i.e., we reduced the complexity of the classifier a half). Shown in Fig. 7(a) and (b) are the result on the overall recognition performance and the total number of parameters in the 4 MLPs respectively.

It is seen from Fig. 7 that the recognition performance was not reduced greatly by using less number of parameters in the 4 MLPs. In fact, the reduction on the recognition performance is very small (0.55% in average which is less than a hundredth) while the reduction on the total number of parameters is large (about one-fourth). This indicates that adaptive assignment of the complexity of a classifier according to the adjustment of its corresponding bit in the TPOC coding process is practically available, and effective. This is beneficial to many largely complicated problems where computation and storage requirements are important factors.

3.6. Measured radar data classification with multiple SVMs

The following results are based on the high-resolution range profiles (HRRPs) of 3 airplane targets, which were measured by radar with the center frequency of 5520 MHz and the bandwidth of 80 Mhz. As shown in Fig. 8, the measured data were segmented. Training and test data were selected from different data segments, among which the 2nd and the 5th segments of Yark-42, the 5th and the 6th segments of An-26, the 6th and the 7th segments of Cessna Citation S/II were taken as the training data, and other data segments were taken as test data. With large SNR, the effect of noise in the measured data was ignored in our experiment. In addition, the measured HRRP is a 52-dimensional vector.

An HRRP is the amplitude of the coherent summations of the complex time returns from target scatterers in each range cell,

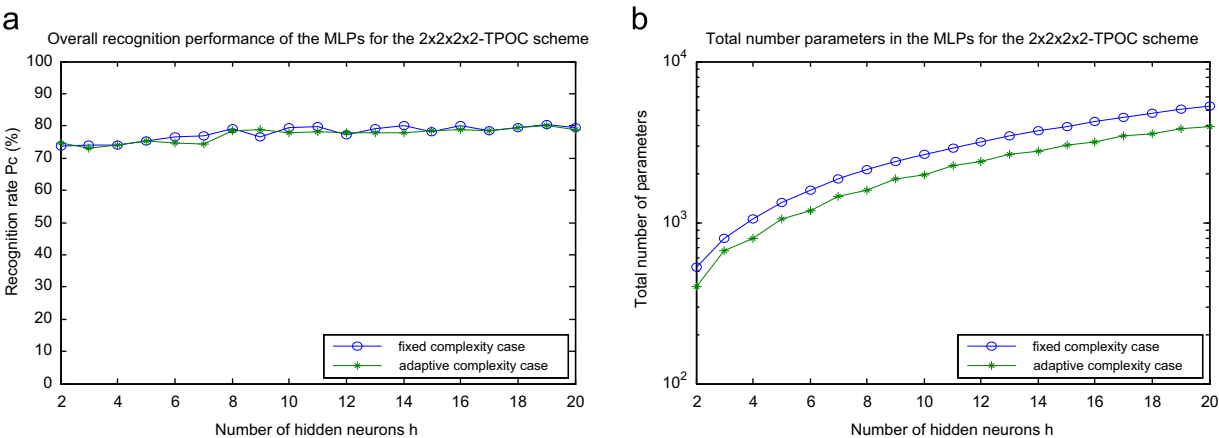


Fig. 7. Performance comparison of the MLPs with fixed complexities and the MLPs with adaptive complexities for the $2 \times 2 \times 2$ -TPOC scheme.

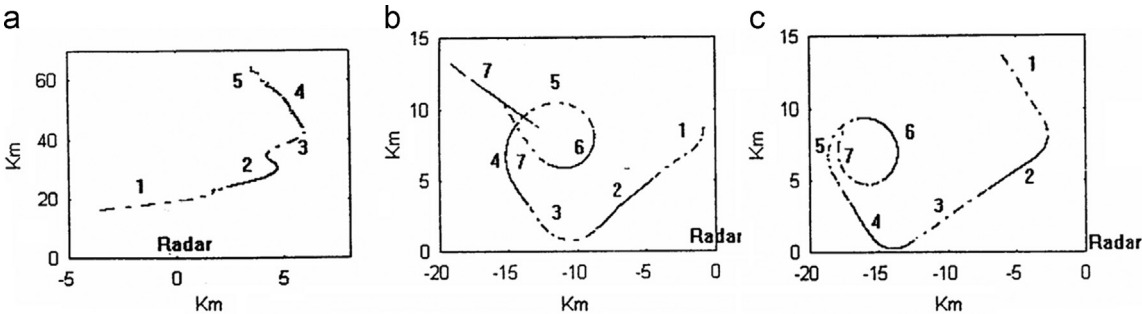


Fig. 8. Projections of target trajectories onto ground plane (the measured data were segmented, and the segments are labeled with integers in the figure): (a) Yark-42, (b) An-26 and (c) Cessna Citation S/II.

Table 4
Recognition results of OAA scheme and TPOC scheme using binary SVMs for the set of measured radar data.

Kernel	RBF kernel, $\sigma^2 = 2$	Polynomial kernel, $d = 3$
OAA-scheme: 3 SVMs		
SVs	856	362
Pc (%)	85.60	84.58
2x2x2 $\times$ 2-TPOC scheme: 4 SVMs		
2x2x2 $\times$ 2-TPOC code	Yark-42 $\rightarrow$ 1010;1110;1111 Cessna $\rightarrow$ 0001;0100;0110 An-26 $\rightarrow$ 0010;0011;1000;1001;1100	
SVs	576	267
Pc (%)	91.62	90.46

which represents the projection of the complex returned echoes from the target scatterer centers onto the radar line-of-sight. It contains the target structure signatures, such as target size, scatterer distribution, etc.; thereby radar HRRP target recognition has received intensive attention from radar automatic target recognition community [32,33]. However, HRRPs are sensitive to target-aspect, amplitude-scale and time-shift [32]. Due to the target-aspect and amplitude-scale sensitivity, HRRPs of one target should be divided into many subsets or HRRP frames according to different angular sectors and amplitude normalized. Du, Liu and Bao [33] showed that power spectrum (the first order spectrum), as a time-shift invariant feature, has the best recognition performance among the higher-order spectra. Therefore, we selected power spectrum feature of target HRRP for the experiment.

According to the aspect angle information, there were 10 HRRP frames for each target. According to the target-aspect sensitivity of HRRP, the RPCL was not used, but the average profile within an

HRRP frame of each target was used as cluster centers of that target in the TPOC scheme. After learning process with the CLA with $t = 80\%$, the average profiles within frames for Yark-42, Cessna Citation S/II and An-26 were discriminatively mapped onto 3, 3 and 5 vertices of a 4-dimensional $2 \times 2 \times 2$ -hypercube respectively, with 5 vertices kept vacant. The code and experimental results are shown in Table 4. Although one more classifier is needed in TPOC scheme compared with the OAA scheme, the total number of support vectors (i.e., 856 or 362) used in the OAA scheme with the radial basis function (RBF) kernel of $\sigma^2 = 2$ or the polynomial kernel of order $d = 3$, is nearly 1.5 times more than that (i.e., 576 or 267) used in TPOC scheme with the same kernel. Furthermore, due to consideration of the target-aspect sensitivity of HRRP, the total recognition rate of our scheme with the radial basis function (RBF) kernel or polynomial kernel is 5–6 percentage larger than that of the OAA scheme with the same kernel. Therefore, TPOC scheme outperforms the OAA scheme in not only less storage requirement, but also better recognition performance.

4. Discussions

Seen from above experiments is great improvement of the TPOC scheme on reducing complexity of an overall classifier with no loss of or even enhanced recognition performance over other coding schemes. Here we add discussions on advantages of the TPOC, the relations to ECOC, and other related approaches.

4.1. Comparison and advantages of TPOC over ECOC

This approach is different from and advantageous over the conventional ECOC approaches in many aspects.

- (1) *Aims are different.* ECOC aims at generating a redundant code for error-correcting ability, while TPOC aims at generating a compact code for an overall classifier of low (intrinsic) complexity without strict definition and measure of complexity.
- (2) *Principles are different.* ECOC attempts to gain error-correcting ability by the addition of redundant bits to an information codeword, while TPOC attempts to gain low (intrinsic) complexity by learning a small sized hypergrid-structured discriminative map of the lowest non-linearity.
- (3) *ECOC is data-independent, while TPOC is data-driven.* In reality, for data-independent code, complexity might be serious, as stated in [3] for ECOC, “in domains where it is important for humans to understand and interpret the induced decision trees, ECOC methods are not appropriate, because they produce very complex trees. In domains where training must be rapid and completely autonomous, ECOC methods with back-propagation cannot be recommended, because of the potential for encountering difficulties during training (local minimum and poor performance on the validation set).” On the contrary, TPOC focuses on achieving the intrinsic complexity of the problem, and hence can be used for problems where it is important for humans to understand and interpret and the domains where training must be rapid and completely autonomous.
- (4) *Low complexity is in principle achieved by the scalability of decomposing a complex classification problem into k -ary simple sub-problems* (k can be different for different sub-problems) rather than only binary sub-problems, or the sub-problem of separating, e.g., only one, two and/or three classes from rest classes in discriminant ECOC[5] and the DECOC[7]. From this view point, the approach is an extension of binary, ternary and binary ECOC code. Furthermore, adaptive assignment of classifier complexities becomes available. These make it highly flexible in using intrinsically complex classifiers for a complicated problem.
- (5) *Error-correcting ability is a natural by-product of the TPOC, and q -TPOC is superior to ECOC under a same user parameter q .* ECOC with parameter q leads the equi-Hamming distances between pairs of codewords to be q ; hence its error-correcting ability is uniform for all classes, equal to $\lfloor (q-1)/2 \rfloor$. On the contrary, q -TPOC with parameter q leads to the *smallest* Hamming distance being at least q ; hence the error-correction ability is *not uniform* for all classes, and is at least $\lfloor (q-1)/2 \rfloor$.

4.2. Relation to other related approaches

This approach maintains or even enhances generalization performance with a less complex classifier. Another approach, referred to as reduced SVM (RSVM) [34–37], also leads to low complexity (less SVs), but with generalization performance loss. The two approaches are in separate directions, one on problem decomposing scheme, while another on only a single classifier (binary SVM). RSVM can be used instead of conventional SVM as a binary base classifier in this approach to further reduce the number of support vectors.

This approach is also different from closely related “ensemble”, “committee” and “boosting” methods [27]. Though all these solve a complex problem by dividing it into a number of simple tasks and then combining the solutions, how many and how simple the classifiers required are different. In boosting, each classifier is too weak (e.g., slightly better than random guess) compared to the intrinsic one of the problem and hence considerably large number of base learners are required. This may not lead to a simple overall classifier. In contrast, in TPOC approach, the complexity approaches the intrinsic one, and the compact code indicates that

fewer classifiers are required, jointly leading the overall classifier to be simple, and easy to learn. Additionally each hypothesis is attempting to predict the same function in ensemble, while it predicts a different function in TPOC. In fact, ensemble approaches can also be applied as an individual base learner in the same manner as we used SVM or MLP in this study.

5. Conclusion

In the context of complicated multi-class tasks, complexity is a main problem. Aiming at decomposing a complex problem into fewer and simpler sub-problems, we contribute the complexity problem in two folds: the TPOC scheme and the CLA for heuristically learning a TPOC map from which the TPOC code is derived.

We proposed a data-driven coding scheme (TPOC scheme) which attempts to gain a compact code and low complexity. The approach is an extension of our previous approach [22] where binary classifiers can only be used as base learners. It is also the first one with complexity, scalability and error-correcting ability dealt with simultaneously. Though classifier complexity is greatly reduced, classification performance is either kept unchanged or even enhanced, the advantage of the TPOC scheme over other coding schemes.

The key problem is to learn a TPOC map from input space to a small sized hypergrid such that both discriminant information preservation (DIP) property and topological order preservation (TOP) property are satisfied: mapping to a small sized hypergrid leads to a compact code, the DIP property guarantees that the code is “meaningful”, and the TOP property is a guarantee that the decision boundary is simple. All these together result in a “meaningful” compact code leading to simple decision boundary.

The distinct advantages of the TPOC scheme is computation efficiency in gaining the code, availability of adaptive assignment on classifier complexity, and no need for strict definition on complexity measure and no need for prior knowledge on statistics of data. The low complexity of the overall classifier achieved from the code reduces the demand of computation and storage resources drastically, which makes it more ready to achieve globally optimal solution of the problem/high generalization performance.

TPOC scheme is based on TPOC map, and CLA is a supervised learning for the map, in which SOM learning algorithm is incorporated for the TOP property. It is different from the LVQ [26,27] which is also supervised learning. LVQ is aimed at refining decision boundary gained from an SOM on clusters [27], which is absolutely different from that of the TPOC. Additionally, TPOC is only hypergrid structured, while LVQ may not be theoretically; the former learns from only cluster centers, while the latter learns from all the data; the former works on only a suitable size of hypergrid which is controlled by parameter t or q , while the latter generally works on a low dimensional mapped space of a large fixed cardinality; LVQ cannot guarantee the DIP property due to the small size of the mapped hypergrid.

In this study, we only tested the scheme via limited and moderate scale problems. Validation of the approach to very large scale problems remains challenging. At first, the structure of the hypergrid (the dimensionality and the cardinality in each dimension) is somewhat trial and error. Secondly, CLA incorporates SOM learning algorithm which is susceptible to local minima especially for the mapped space of high dimensionality [38]. Furthermore, boundary complexity also depends on scattering of samples in clusters. Hence, search for globally (sub-)optimal solution of the SOM and extending the SOM learning algorithm to a higher dimensional hypergrid within acceptable computation and storage

requirements are two separate challenging directions for the CLA to be practical for very large scale problems.

Further work includes validation to more datasets, more types of classifier as a base learner (e.g., decision tree and/or naïve Bayes classifier), and effect of the hypergrid structure on decomposition performance, as well as robustness of the gained classifier. Learning algorithm for a TPOC map without incorporation or with other incorporation form of the SOM learning algorithm is encouraged. Relation between the TPOC code and the two separation properties of rows and columns in ECOC code should be studied for further understanding the relation between the TPOC and ECOC. A more exact mathematical interpretation especially from the context of complexity measure should be studied. Furthermore, a promising direction for improving the proposed scheme is designing a code which captures the inherent but possibly hierarchical “clumpiness” of the data, without loss of code scalability. It might also be promising to extend the approach to continuous code [40]. Jointly learning the coding and classifiers rather than learning them separately is an open problem for getting an overall classifier of low complexity.

Acknowledgments

The authors are very grateful to the anonymous referees for their careful reading of this paper and valuable and constructive comments, which help to improve the quality and the readability of the paper. Also, we acknowledge the generous support of the Chinese National Science Foundation under Grant nos. 61070137, 91130006 and 60933009.

Appendix A1. Coding classes from a TPOC map

For a K -class problem, a TPOC map from t cluster centers to an m -dim $p_1 \times p_2 \times \dots \times p_m$ -hypergrid $Z_{p_1} \times Z_{p_2} \times \dots \times Z_{p_m}$, where $Z_{p_k} = \{1, 2, \dots, p_k\}$, $k = 1, 2, \dots, m$, can be expressed by

$$\Omega: \{1, 2, \dots, K\} \rightarrow Z_{p_1} \times Z_{p_2} \times \dots \times Z_{p_m}.$$

We have two available intuitive code matrices for coding classes, k -ary code matrix M and binary code matrix $\tilde{M}$.

k-ary coding scheme: using k-ary classifiers

Each dimension of the hypergrid corresponds to a k -ary classifier, and hence altogether m classifiers are used. The i th classifier, $i = 1, 2, \dots, m$, which corresponds to the i th dimension of the hypergrid, is therefore a p_i -ary classifier. Simply denoted as $1-2-\dots-p_i$, it separates samples belonging to $* \dots * 1 * \dots *$, $* \dots * 2 * \dots *$, and $* \dots * p_i * \dots *$ (the i th bit in the string is $1, 2, \dots, p_i$ in each string respectively, while leave all the other bit to be not-care). Hence, the code matrix M is $K \times m$, and its j th element of the i th column of the $K \times m$ code matrix M is

$$M_{ji} = l \quad \text{if } \Omega(j) = * \dots * l * \dots *$$

for $j = 1, 2, \dots, K$, or equivalently, the j th row of the code matrix is $q_{j1}q_{j2}\dots q_{jm}$ if we have $\Omega(j) = q_{j1}q_{j2}\dots q_{jm}$.

Binary coding scheme: using binary classifiers

In this scheme, each k -ary classifier $1-2-\dots-k$ in the k -ary coding scheme can be substituted by $k-1$ binary classifiers, i.e., the one separating 1 from $2, 3, \dots, k$, and the one separating 1, 2, from $3, 4, \dots, k$, and the one separating $1, 2, \dots, k-1$ from k . Hence, the corresponding column of the code matrix M in the k -ary coding scheme, say $[q_{1j}, q_{2j}, \dots, q_{Kj}]^T$, is substituted with $k-1$ columns in this scheme: its

l th column separates $1, 2, \dots, l$ from $l+1, \dots, k$, and therefore its i th element is

$$\tilde{M}_{il} = \begin{cases} 0 & \text{if } q_{ij} \leq l \\ 1 & \text{otherwise} \end{cases} \quad \text{for } i = 1, 2, \dots, K$$

Altogether $\sum_{j=1}^m (p_j-1)$ binary classifiers are required in this scheme.

References

- [1] Bart L.M. Happel, Jacob M.J. Murre, Design and evolution of modular neural network architectures, *Neural Networks* 7 (6) (1994) 985–1004.
- [2] V. Ramamurti, J. Ghosh, Structurally Adaptive Modular Networks for Nonstationary Environments, *IEEE Trans. Neural Networks* 10 (1) (1999) 152–160.
- [3] Daniel J. Sebald, James A. Bucklew, Support vector machines and the multiple hypothesis test problems, *IEEE Trans. Signal Process.* 49 (11) (2001) 2865–2872, November.
- [4] Thomas G. Dietterich, Ghulam Bakiri, Solving multiclass learning problems via error-correcting output codes, *J. Artif. Intell. Res.* 2 (1995) 263–286.
- [5] O. Pujol, P. Radeva, J. Vitri'a, Discriminant ECOC: a heuristic method for application dependent design of error correcting output codes, *IEEE Trans. PAMI* 28 (2006) 1007–1012.
- [6] E. Alpaydin, E. Mayoraz, Learning error-correcting output codes from data, in: *Proceedings of International Conference on Artificial Neural Networks (ICANN'99)*, vol. 2, 1999, pp. 743–748.
- [7] Jie Zhou, Hanchuan Peng, Ching Y. Suen, Data-driven decomposition for multiclass classification, *Pattern Recognition* 41 (1) (2008) 67–76.
- [8] Francesco Masulli, Giorgio Valentini, Effectiveness of error correcting output coding methods in ensemble and monolithic learning machines, *Pattern Anal. Appl.* 6 (4) (2003) 285–300.
- [9] Tomonori Kikuchi, Shigeo Abe, Comparison between error correcting output codes and fuzzy support vector machines, *Pattern Recognition Lett.* 26 (12) (2005) 1937–1945, September.
- [10] Aldebaro Klautau, Nikola Javtic, Alon Orlisky, On nearest-neighbor error-correcting output codes with application to all-pairs multiclass support vector machine, *J. Mach. Learn. Res.* 4 (2003) 1–5.
- [11] Sergio Escalera, Oriol Pujol, Petia Radeva, Separability of ternary codes for sparse designs of error-correcting output codes, *Pattern Recognition Lett.* 30 (3) (2009) 285–297.
- [12] Dijun Luo, Rong Xiong, An improved error-correcting output coding framework with kernel-based decoding, *Neurocomputing* 71 (16–18) (2008) 3131–3139.
- [13] Elif Derya Übeyli, Time-varying biomedical signals analysis with multiclass support vector machines employing Lyapunov exponents, *Digital Signal Process.* 18 (4) (2008) 646–656.
- [14] W. Xi Long, Louis Cleveland, Y. Lawrence Yao, Multiclass cell detection in bright field images of cell mixtures with ECOC probability estimation, *Image Vision Comput.* 26 (4) (2008) 578–591.
- [15] Elif Derya Übeyli, Analysis of EEG signals by combining eigenvector methods and multiclass support vector machines, *Comput. Biol. Med.* 38 (1) (2008) 14–22.
- [16] G.B.T.G. Dietterich, Achieving high-accuracy text-to-speech with machine learning, *Data Mining Speech Synth.* (1999).
- [17] Koby Crammer, Yoram Singer, Ultraconservative online algorithms for multiclass problems, *J. Mach. Learn. Res.* 3 (2003) 951–991.
- [18] Yu-Chieh Wu, Yue-Shi Lee, Jie-Chi Yang, Robust and efficient multiclass SVM models for phrase pattern recognition, *Pattern Recognition* 41 (9) (2008) 2874–2889.
- [19] Elif Derya Übeyli, Doppler ultrasound signals analysis using multiclass support vector machines with error correcting output codes, *Expert Syst. Appl.* 33 (3) (2007) 725–733.
- [20] Elif Derya Übeyli, Multiclass support vector machines for diagnosis of erythematous-squamous diseases, *Expert Syst. Appl.* 35 (4) (2008) 1733–1740.
- [21] Sergio Escalera, Oriol Pujol, Petia Radeva, Boosted landmarks of contextual descriptors and forest-ECOC: a novel framework to detect and classify objects in cluttered scenes, *Pattern Recognition Lett.* 28 (13) (2007) 1759–1768.
- [22] Lan Du, Junying Zhang, Zheng Bao, Multi-class classification based on the hypercube self-organizing mapping (SOM) scheme, the second International Conference on Natural Computation (ICNC'06) and the third International Conference on Fuzzy Systems and Knowledge Discovery (FSKD'06), Xi'an, China from 24–28 September, *Lecture Notes Comput. Sci.* 2006 (2006) 107–110.
- [23] K. Tumer, N.C.Oza, Decimated input ensembles for improved generalization, in: *Proceedings of the International Joint Conference on Neural Networks*, Washington, DC, 1999.
- [24] Y. Freund, R.E. Schapire, Experiments with a new boosting algorithm, in: *Proceedings of the 13th International Conference on Machine Learning*, Morgan Kaufman, 1996, pp. 148–156.
- [25] L. Breiman, Bagging predictors, *Mach. Learn.* 24 (2) (1996) 123–140.
- [26] Anil K. Jain, Robert P.W. Duin, Jianchang Mao, Statistical pattern recognition: a review, *IEEE Trans. Pattern Anal. Mach. Intell.* 22 (1) (2000) 4–37.
- [27] Simon Haykin, *Neural Networks: A Comprehensive Foundation*, 2nd edition, Prentice-Hall, Inc, 1999.

- [28] S. Escalera, O. Pujol, P. Radeva, Loss-weighted decoding for error-correcting output codes, in: *Proceedings of the International Conference on Computer Vision Theory and Applications*, vol. 2, 2008, pp. 117–122.
- [29] J. Khan, J.S. Wei, M. Ringner, et al., Classification and diagnostic prediction of cancers using gene expression profiling and artificial neural networks, *Nat. Med.* 7 (6) (2001) 673–679.
- [30] Hanchuan Peng, Fuhui Long, Chris Ding, Feature selection based on mutual information: criteria of max-dependency, max-relevance, and min-redundancy, *IEEE Trans. Pattern Anal. Mach. Intell.* 27 (8) (2007) 1226–1238.
- [31] Marco Loog, R.P.W. Duin, Multiclass linear dimension reduction by weighted pairwise Fisher criteria, *IEEE Trans. Pattern Anal. Mach. Intell.* 23 (7) (2001) 762–766.
- [32] R.A. Mitchell, R. Dewall, Overview of high range resolution radar target identification, in: *Proceedings of Automatic Target Recognition Working Group*, Monterey, CA, 1994.
- [33] L. Du, H.-W. Liu, Z. Bao, Radar HRRP target recognition based on higher-order spectra, *IEEE Trans. Signal Process.* 53 (7) (2005) 2359–2368.
- [34] DucDung Nguyen, TuBao Ho, An efficient method for simplifying support vector machines, in: *Proceedings of the 22nd International Conference on Machine Learning*, Bonn, Germany, 2005.
- [35] Bernhard Schoelkopf, Sebastian Mika, Chris J C Burges, Philipp Knirsch, Klaus-Robert Mueller, Gunnar Raetsch, Alexander J Smola, Input space versus feature space in Kernel-based methods, *IEEE Trans. Neural Networks* 10 (5) (1999) 1000–1017.
- [36] C.J.C. Burges, Simplified support vector decision rules, in: L. Saitta, (Ed.), *Proceedings of the 13th International Conference on Machine Learning*, Morgan Kaufmann, 1996, pp. 71–77.
- [37] Kuan-Ming Lin, Chih-Jen Lin, A study on reduced support vector machines, *IEEE Trans. Neural Networks* 14 (6) (2003) 1449–1459.
- [38] Tom Heskes, Energy functions for self-organizing maps, *Kohonen Maps* (1999) 303–315.
- [39] L. Xu, A. Krzyżak, E. Oja, Rival penalized competitive learning for clustering analysis, RBF net, and curve detection, *IEEE Trans. Neural Networks* 4 (4) (1993) 636–648.
- [40] Koby Crammer, Yoram Singer, On the learnability and design of output codes for multiclass problems, *Mach. Learn.* 47 (2002) 201–233.



Xiaoxue Zhao is a PhD candidate in the Department of Computer Science, University College London, London, UK, from 2011 to present. She received her Bachelors degree and MS degree in the Department of Physics, Fudan University, Shanghai, China, in 2008 and 2011 respectively. She received the first place scholarship named “Morgan Stanley Scholarship” at the second year in university. She was a visiting research student, invited by the Department of Applied Physics, Hokkaido University, Japan, in 2009 and in 2010, and she was a visiting research student, supported by GCOE-BIEP exchange program (Global Centers of Excellence-Bilateral International Exchange Program), at Department of Physics, Kyoto University, Japan, in 2009. Her research interests are in machine intelligence, information retrieval, data mining, agent-based modeling, and statistical data analysis.



Lan Du received her B.Eng., M.Eng. and Ph.D. degrees in Electronic Engineering Institute from Xidian University, Xi'an, China, in 2001, 2004 and 2007 respectively. She is now a professor in the institute. Her research interests are in the fields of radar automatic target recognition (RATR), radar imaging (SAR/ISAR) and radar signal processing.



Junying Zhang received her PhD degree in 1998 in Signal and Information Processing from Xidian University, Xi'an, China. From 2001 to 2002, she was a visiting scholar at the department of electrical engineering and computer science, the Catholic University of America, Washington, DC, USA, and in 2007, she was a visiting professor at the department of electrical engineering and computer science, Virginia Tech University, USA. She is currently a professor in the School of Computer Science and Technology, Xidian University, Xi'an, China. Her research interests focus on intelligent information processing, including machine learning and its application to cancer related bioinformatics, causative learning and pattern discovery.